CHRIST (Deemed to be University)

B.Tech • Computer Science and Engineering

CASE STUDY REPORT

Adaptive Cloud Server Resource Allocation

A Reinforcement Learning Formulation

Reinforcement Learning | CIA–3 | Component 1

September 2026

Submitted by

Team member 1: _____ Register no.: _____

Team member 2: _____ Register no.: _____

Team member 3: _____ Register no.: _____

Team member 4: _____ Register no.: _____

Submitted to: _____

Scope: proposed case study and RL formulation. Implementation and comparative results belong to Component 2.

Abstract and contents

Cloud applications experience changing request volumes, making a fixed server allocation inefficient. Insufficient capacity increases queueing delay and service failures, while excessive capacity raises operating cost. This case study proposes a reinforcement learning controller for horizontal resource allocation in a simulated cloud application. A centralized agent chooses whether to add one instance, remove one idle instance, or maintain the current allocation at each control interval. Its compact observation contains request-rate and queue bands, active instance count, pending startup status, and recent demand trend. A normalized reward encourages timely request completion and penalizes resource expenditure, service failures, backlog, and frequent scaling. Q-learning is selected as the initial method because the restricted state and action spaces permit a small, inspectable value table. The study identifies partial observability, startup delay, uncertain workload, and constrained actions as central modelling issues. A workload-forecasting extension is proposed to support earlier scaling before predictable demand increases. The report also outlines a fair simulation-based comparison of four related algorithms for Component 2. All numerical settings and worked examples are design assumptions; no experimental improvement is claimed.

Keywords: reinforcement learning; cloud autoscaling; resource allocation; Q-learning; queueing; workload prediction.

Contents

1. Problem definition and scope

1.1 Real-world problem

Consider a cloud-hosted web application that accepts search queries, retrieves records and returns responses through an API. The volume of incoming requests changes throughout the day and can rise suddenly when many users access the application together. Requests must share a finite pool of compute instances. When demand exceeds available processing capacity, work accumulates and users wait longer. Keeping many instances active during quiet periods reduces this risk but consumes resources that are not required.

The resource-allocation problem is to decide how much capacity to keep available as conditions change. This report focuses on horizontal scaling: adjusting the number of identical application instances. It does not change the CPU or memory assigned to an individual instance. This restricted scope makes the relationship between allocation, service quality and cost visible and supports a manageable simulator for later implementation.

1.2 Decision maker and stakeholders

The agent is a centralized cloud resource manager. It observes workload and capacity telemetry and issues one allocation decision per minute. The environment is the application-serving system: request arrivals, the waiting queue, active servers and pending startup operations. Users benefit from timely responses; the application operator benefits from lower infrastructure expenditure and predictable performance. These interests can conflict, so a useful policy must consider them together.

1.3 Research question and objective

The research question is: can a compact reinforcement learning controller learn a scaling policy that balances response-time compliance and resource cost under changing demand? The intended objective is to maximize cumulative discounted reward over repeated decisions while respecting capacity and action constraints. Reward includes service success, resource use, service failures, queue backlog and changes in allocation. This establishes a measurable decision problem rather than treating low CPU usage or a low server count as sufficient evidence of success.

1.4 Case-study boundary

The proposed service target is a 95th-percentile response time of at most 200 milliseconds, with failures reported separately. This threshold is a project assumption, not a contractual promise from a cloud provider. The system has one application, one resource pool and between one and five identical instances. Database scaling, geographic placement, network outages, heterogeneous hardware and real cloud billing rules are outside the initial model. The selected topic continues into Component 2 as required by the assessment brief [1].

2. Sequential decisions and the baseline

2.1 Why this is a sequential problem

A scaling action changes the conditions under which later requests are served. Adding a server increases available capacity only after startup; removing one reduces future capacity and can leave the remaining servers overloaded. Work already waiting in the queue persists across decision boundaries. Consequently, the best action depends on more than the latest request count. The controller must account for existing backlog, recently requested capacity and the likely persistence of current demand.

Suppose two instances can collectively process about 200 requests per second, but demand rises to 240. Requesting a third instance does not immediately eliminate the shortfall. During its startup interval, the original two instances still serve the workload. Once the third instance is ready, the additional capacity can process new arrivals and any remaining backlog. This example demonstrates delayed consequences; it is capacity arithmetic, not a trained policy or a measured latency result.

2.2 Real-time decisions at a practical cadence

The cloud service runs continuously, while the controller acts every 60 seconds. This is real-time sequential control at a defined cadence, rather than a hard real-time guarantee. The simulator should maintain finer-grained request timestamps because a one-minute aggregate cannot directly reveal whether an individual request exceeded 200 milliseconds. Policy decisions are comparatively infrequent; service events and queue updates occur between them.

2.3 What a rule-based system provides

A useful baseline scales out when CPU utilization remains above a chosen threshold and scales in when it remains below another threshold. A stabilization or cooldown mechanism reduces frequent changes. Existing Kubernetes autoscaling supports observed metrics, replica limits and stabilization behaviour [2]. Therefore, the comparison must use a reasonably configured baseline, not an intentionally weak rule with no delay handling.

A fixed threshold rule does not directly learn how much future service improvement justifies an additional minute of resource cost in this simulator. Its thresholds may require adjustment when workload patterns change. RL is proposed to learn this trade-off from repeated experience. It is not assumed to outperform a well-tuned rule: any benefit must be demonstrated on unseen workloads.

2.4 Short-term and long-term objectives

The immediate objective is to serve the current interval without unnecessary expenditure. The longer-term objective is to avoid repeated overload, persistent backlog and cycles of scaling out and back in. A short period of additional capacity may be worthwhile if it prevents later delays. Conversely, retaining extra capacity after demand has fallen wastes resources. Discounted return connects these consequences across time.

3. Environment and simulator assumptions

3.1 Proposed system model

Incoming requests enter a shared first-in, first-out queue. Each active instance serves one request at a time and takes another queued request when it becomes available. For the initial simulator, service durations are sampled independently with a mean of 10 milliseconds, corresponding to an approximate mean service capacity of 100 requests per second per instance. Exponentially distributed service times are a simplifying assumption; sensitivity experiments should later test less variable and more variable service times.

Arrivals are generated from a rate process that can represent quiet periods, gradual ramps and bursts. A time-varying Poisson arrival process is a starting assumption, not a claim about all cloud traffic. The workload generator knows the underlying rate schedule; the controller receives only measurements from elapsed intervals. This prevents access to future traffic information during ordinary control.

Table 1. Initial simulator configuration; all values are proposed.

Parameter	Initial setting
Control interval	60 seconds
Instance bounds	1 minimum; 5 maximum, including pending capacity
Mean service time	10 ms per request per instance
Startup delay	One full control interval before usable capacity
Waiting queue limit	6,000 requests; overflow requests are rejected
Request timeout	10 seconds from arrival, including service time
Service target	95th-percentile completed response time ≤ 200 ms

3.2 Outcomes and accounting

A request is resolved when it completes, is rejected, or reaches its timeout. A timed-out request is removed and its remaining work is cancelled in the initial model. Every resolved request is counted exactly once. Response time includes queueing and service, but excludes network delay. Memory is fixed and assumed sufficient. Active and starting instances both accrue normalized resource cost; the one-instance minimum keeps service capacity available.

A full request-level event log is retained for evaluation. Aggregate observations support decisions, while detailed logs support response-time percentiles, failure counts and inspection of unusual episodes. This separation keeps the controller small without sacrificing measurement accuracy. The design does not require access to paid cloud infrastructure.

4. State representation

4.1 Compact observation for the controller

The Q-table uses a discrete observation s_t = (bλ, bq, n, p, d). These components summarize demand, backlog, ready capacity, pending startup and recent trend. Continuous telemetry is binned to keep the table small. The simulator retains richer internal information, including request ages, but does not expose all of it to the learning agent.

Table 2. State variables and proposed discretization.

Variable	Encoding	Decision relevance
Arrival band bλ	Low: <150 req/s; medium: 150–300; high: >300	Recent request demand
Queue band bq	Empty: 0; short: 1–500; long: >500	Unfinished waiting work
Active count n	Integer from 1 to 5	Ready capacity and cost
Pending status p	0 or 1	Capacity awaiting activation
Trend d	Falling, flat or rising	Recent demand direction

4.2 Trend and state-space size

Trend compares the two most recent interval arrival rates. Define the relative change as (latest rate − previous rate) / max(previous rate, 1 request/second). Values below −10% are falling, above +10% are rising, and the rest are flat. At initialization, use flat until two measurements exist. Bin edges must remain unchanged across the compared algorithms and be fitted without consulting evaluation workloads.

$$\text{Nominal states} = 3 \times 3 \times 5 \times 2 \times 3 = 270$$

With three actions, the initial table contains at most 810 state–action values. Some combinations are impossible, such as five active instances plus a pending sixth instance. CPU utilization and latency remain available for monitoring and reward calculation; they are not additional dimensions in this first table. This preserves the formulation used in the presentation.

4.3 Information loss and its consequence

Different detailed situations can share the same binned observation. Two long queues, for example, may contain requests with very different ages and timeout risks. The controller also does not observe the workload generator's hidden regime. Therefore, this is an approximate state representation for a partially observable system. A compact table is tractable, but it cannot guarantee that every relevant future consequence is recoverable from the current observation alone.

5. Actions, constraints and transitions

5.1 Discrete action space

At each decision boundary, the agent selects a valid action from A = {−1, 0, +1}. The symbols represent removing one idle instance, holding the allocation, and requesting one new instance. Restricting changes to one instance per decision prevents abrupt jumps in capacity and makes the learned choices easy to interpret.

Table 3. Action effects and validity rules.

Action	Effect	When permitted
+1: scale out	Start one additional instance	No startup pending; n + p < 5
0: hold	Keep allocation; allow scheduled activation	Always
−1: scale in	Remove one idle instance immediately	No startup pending; n > 1; an idle instance exists

Invalid actions are masked both during random exploration and greedy selection. The simulator supplies the valid-action mask, including whether an idle instance is currently available. This mask provides additional operational information beyond the five-variable observation. It is applied identically to all algorithms. In-flight work is not discarded to implement a scale-in action.

5.2 Startup timing convention

An instance requested at boundary t remains unavailable throughout interval t and becomes usable in interval t + 1. To keep pending status meaningful, the observation at the next boundary is taken immediately before its scheduled activation. It still reports the old active count and p = 1. Only hold is allowed at that boundary; the ready instance then activates before processing the next interval. This explicit ordering reconciles one-step startup delay with the pending-startup observation.

5.3 Transition from one observation to the next

After selecting an action, the simulator applies any permitted removal or new startup and processes the interval's arrivals and service events. It records completed, rejected and timed-out requests, integrates billed instance time, and samples the queue at the boundary. The next arrival band and trend use the interval that has just finished. The returned reward therefore describes the consequences of the chosen action over an elapsed interval, not an estimate of unseen future outcomes.

The underlying environment may be written as a controlled stochastic process, with transitions depending on capacity, outstanding work and random arrivals and service durations. Its full internal state includes information omitted by the compact observation. Transition probabilities need not be known by the Q-learning controller. The event-order convention and mask rules should be documented in code so all later algorithms solve the same task.

6. Reward design and worked examples

6.1 A normalized multi-objective reward

The reward is computed after each control interval. Let R be the number of requests resolved in that interval. Let G count completions within 200 ms; V counts later completions, rejections and timeouts. Set g = G/R and v = V/R when R > 0; otherwise set both to zero. Successful and failed outcomes partition resolved requests, so they are not counted twice within either category.

$$r_{t+1} = 2g - c - 4v - q - 0.2m$$

Here c is billed active-plus-starting instance-seconds divided by 5 × 60, the maximum permitted resource use per interval. The queue term q is min(queue length / 6,000, 1). The change indicator m equals one when an agent-requested scale-in or scale-out is applied, and zero for hold; scheduled activation is not charged as a second action. Thus c, q and m are bounded and measured consistently.

6.2 Positive and negative outcomes

The positive term rewards timely resolution, while the cost term discourages unnecessary capacity. Failures receive a larger weight because cheap service is not useful if requests cannot be served. Queue and action-change penalties discourage allowing work to accumulate and repeatedly reversing decisions. These weights are proposed preferences rather than experimentally tuned values. A reward penalty encourages service compliance but does not enforce the 200 ms target as a hard guarantee.

Table 4. Illustrative interval rewards; these are not training results.

Scenario	Inputs (g, c, v, q, m)	Reward
Efficient service	0.98, 0.40, 0.02, 0.05, 0	1.43
Same service, extra cost	0.98, 0.80, 0.02, 0.05, 0	1.03
Overload despite lower cost	0.70, 0.40, 0.30, 0.60, 0	−0.80

For the first row: 2(0.98) − 0.40 − 4(0.02) − 0.05 = 1.43. The second row shows that equal service quality with greater resource use receives less reward. If an allocation change were applied in either interval, another 0.20 would be deducted.

6.3 Reward limitations

Resolved-request fractions can delay the appearance of failures while work remains queued. Backlog penalties and a finite timeout reduce this blind spot. Evaluation must also inspect failure fraction and latency independently of reward. Sensitivity analysis should vary the cost, failure and switching weights, since a policy can appear successful simply because the chosen weights strongly favour its behaviour.

7. RL problem characteristics

The service continues indefinitely, while training uses finite windows. Stochastic transitions and partial observability are different properties: missing information, rather than randomness alone, causes partial observability here.

Table 5. Characteristics of the proposed RL formulation.

Dimension	Classification	Reason
Task horizon	Continuing	No natural terminal state.
Environment	Dynamic	Load and capacity evolve.
Transitions	Stochastic	Random arrivals and service.
Observability	Partial	Hidden queue ages and demand regime.
State/action space	Discrete	270 states; three candidate actions.
Decision structure	Sequential	Delayed capacity and queue effects.
Learning setting	Simulation-based	Exploration in a custom simulator.

7.1 Training windows are not natural termination

A training run may be divided into fixed windows, for example 120 control steps, to organize experiments. The end of such a window is a time-limit truncation, not evidence that future value becomes zero. For a continuing formulation, the learning target should bootstrap from the final next observation rather than force it to zero solely because a window ended. Reset behaviour, warm-up periods and initial queue conditions must be consistent across algorithms.

7.2 Online interaction within an offline simulator

During training, the agent interacts step by step with a simulator and updates its table from fresh transitions. In this report, simulation-based learning describes where that interaction occurs. It is distinct from training exclusively on a fixed historical dataset. Evaluation uses a frozen table and no exploratory actions. The report does not propose exploratory learning directly on a live customer-facing service.

7.3 Stationarity and generalization

A dynamic workload can still be generated by a fixed stochastic rule; changing observations alone do not prove that the transition law is nonstationary. A deliberate shift in the arrival generator or service distribution is a separate test of adaptation or robustness. Training on multiple patterns may help coverage, but performance on a familiar traffic shape does not establish that the policy generalizes to every unseen workload.

8. Algorithm selection: Q-learning

8.1 Why Q-learning fits the initial study

Q-learning estimates the value of taking an action in an observed state using sampled rewards and a greedy next-state target. It is a model-free, off-policy control method [3]. For this case study, the 270-state abstraction and three actions yield a small table that can be inspected directly. A table is also easy to compare with the behaviour of a threshold baseline: for a rising arrival band and long queue, the preferred valid action can be read without interpreting a neural network.

An exact dynamic-programming solution would need a transition and reward model over the chosen state representation. A deep model adds complexity that is not necessary for the initial table size. Q-learning is therefore a suitable first implementation, although its suitability is a design choice rather than proof that it will be the best-performing approach.

8.2 Update rule

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha\,[r_{t+1} + \gamma \max_{a'} Q(s_{t+1}, a') - Q(s_t, a_t)]$$

The maximization considers only valid next actions. The learning rate α controls the influence of a new observation, while γ discounts future rewards. A proposed starting configuration is α = 0.10 and γ = 0.95. At a one-minute cadence, γ = 0.95 gives material weight to consequences over several later intervals. These values should be tuned on training and validation workloads only.

8.3 Exploration and a worked update

Use ε-greedy exploration: with probability ε choose uniformly among valid actions; otherwise choose an action with the largest value, breaking ties consistently. A proposed schedule decreases ε from 1.00 to 0.05 over the first 80% of the allocated training steps, then holds it at 0.05. Evaluation sets ε to zero and freezes the table. All methods should receive an equivalent exploration budget.

For a hypothetical transition, suppose Q(s,a) = 0.50, reward = 1.43 and the largest valid next-state value is 0.80. The target is 1.43 + 0.95(0.80) = 2.19. The updated value becomes 0.50 + 0.10(2.19 − 0.50) = 0.669. This arithmetic explains the update; the values are not learned measurements.

8.4 Limits of the theoretical guarantee

Classical tabular convergence results require conditions including a Markov setting, adequate repeated sampling, bounded rewards and an appropriate learning-rate sequence [3]. The present binned observation is partially observable, and a constant starting learning rate is a practical experiment setting. Consequently, the report makes no claim of guaranteed optimality or convergence for this cloud formulation.

9. End-to-end RL workflow

9.1 Environment to state

Initialize a workload seed, queue, active count and pending status. During a warm-up interval, collect enough arrival measurements to define the initial observation. At each decision boundary, construct the five-variable state from elapsed telemetry. Determine the valid-action mask from instance limits, pending startup and idle-instance availability. Keep hidden workload schedules and future service samples outside the agent interface.

9.2 State to agent to action

Look up the current Q-values and select a valid action using the exploration rule. An action of +1 requests an additional instance; −1 removes an idle instance; 0 retains the allocation. If pending activation is reported, hold is the only available action and the instance activates according to the ordering in Section 5. The controller does not repeatedly request the same pending capacity.

9.3 Action to reward and next state

Process the next 60 seconds of simulated time using request-level events. Record successful completions, late completions, rejections and timeouts, then integrate billed instance time and measure boundary backlog. Compute the normalized reward and encode the next observation. The environment returns the transition $(s_t, a_t, r_{t+1}, s_{t+1})$, the next valid-action mask and whether the run reached a time limit.

9.4 Learning and repetition

Apply the Q-learning update once for the observed transition. Continue collecting experience until the training budget ends, reducing ε according to the fixed schedule. Save the table, configuration and workload seeds together. During evaluation, use the same observation and action machinery with learning disabled. This clean separation ensures that evaluation results describe a fixed policy rather than a policy that continues to adapt to its test workload.

9.5 Illustrative burst walkthrough

Before a burst, two instances provide approximately 200 requests/second of mean capacity against 120 requests/second of demand. Holding two instances is one possible decision; whether scaling in is worthwhile depends on queue and service outcomes. When demand reaches 240 requests/second, capacity is insufficient on average. If the chosen action is scale out, the new instance starts while the current interval continues with two active instances.

At the following boundary, pending activation forces hold and the new server becomes usable. Approximate mean capacity increases to 300 requests/second, providing headroom over continuing demand of 240. Remaining queued work can be served, although already missed deadlines cannot be recovered. Actual response times depend on arrival and service samples. This is an illustrative workflow, not a learned result.

10. Innovation: RL with workload prediction

10.1 Proposed extension

The innovation adds a short-horizon workload forecast to the existing RL controller. The rationale is specific to startup delay: when demand is likely to rise next interval, waiting until the queue is already long may be too late to protect early requests. The forecast does not dictate an action. It supplies another observation from which the agent can learn whether earlier capacity is worth its cost.

Begin with exponential smoothing rather than a complex forecasting model. After observing arrival rate λ_t for the interval that has just elapsed, update the estimate $F_{t+1} = \beta\lambda_t + (1 - \beta)F_t$, where β is selected using training and validation data. Initialize the forecast from the first measured rate. A β of 0.30 is an illustrative starting choice, not a tuned result. No future request count may enter the forecast.

10.2 Integration into the controller

Convert the forecast into low, medium or high using the same arrival-band boundaries as Section 4, and append it to the observation. The extended state becomes (bλ, bq, n, p, d, bF). Its nominal size is 270 × 3 = 810 states, producing at most 2,430 action values. The action space, startup convention, capacity limits and reward remain unchanged. The same limits also apply when a forecast is incorrect.

The original trend variable indicates recent direction, whereas the forecast estimates the next level of demand. The two signals may overlap, so their value should be assessed experimentally. A useful supplementary comparison can remove the trend term from the forecast version to determine whether extra state dimensions provide useful information or merely increase the number of observations required for learning.

10.3 Evaluation of the innovation

Compare ordinary Q-learning with forecast-augmented Q-learning under matching training budgets, reward weights, seeds and held-out workload traces. Include gradual ramps, recurring peaks and abrupt unannounced bursts. The first two patterns are more plausibly predictable; a smoothing forecast cannot reliably anticipate a burst with no precursor. Report forecast error alongside policy cost and service outcomes to connect prediction quality to its operational effect.

10.4 Expected value and limitations

The hypothesis is that prediction may reduce startup-related service failures on predictable demand increases. It may also cause unnecessary allocation when a predicted increase does not occur. Because the extended table is larger, any improvement must justify additional training and complexity. This is a proposed RL-plus-predictive-analytics extension consistent with the assessment's innovation requirement [1]; it is not a claim that a new forecasting algorithm has been developed.

11. Evaluation plan and conclusion

11.1 Continuation into Component 2

Implement Q-learning, SARSA, Expected SARSA and Dyna-Q in one project using the same simulator. SARSA uses its selected next action in the target; Expected SARSA averages over the next-action policy; Dyna-Q adds updates from a learned transition model [4]. Include the threshold autoscaler as a non-RL baseline. Train on a shared workload distribution and evaluate frozen policies on the same held-out traces across multiple random seeds.

Table 6. Proposed metrics and definitions.

Metric	Calculation or interpretation
Mean reward per step	Sum of interval rewards divided by evaluated control steps.
Resource cost	Total active-plus-starting instance-seconds over the evaluation horizon.
Failure fraction	Late completions + rejections + timeouts, divided by all arrivals in the evaluated cohort.
95th-percentile response time	95th percentile of completed-request response times; report failures alongside it.
Training time and variability	Wall-clock training time; mean and spread of each metric across seeds.

Resolve every request in the evaluated arrival cohort before computing final failure statistics; use a documented drain period so end-of-window backlog is not silently ignored. Keep training budgets comparable and disclose Dyna-Q planning updates. Compare both interaction budget and runtime because extra model updates require computation. Plot reward learning curves and capacity, traffic and queue traces; do not select a winner using training reward alone.

11.2 Limitations and responsible interpretation

The model omits network delay, shared-database bottlenecks and hardware heterogeneity. Binned observations lose detail, and reward weights can strongly influence behaviour. A policy trained on synthetic workloads may fail under unseen patterns. A cost reduction is useful only when accompanied by acceptable service outcomes. Any later deployment would require validation against realistic traces and monitoring beyond this classroom simulator.

11.3 Conclusion

Adaptive cloud resource allocation forms a clear sequential decision problem: present capacity choices affect later queues, response times and expenditure. This case study defines a tractable environment, compact state, constrained actions, normalized reward and Q-learning workflow. Workload prediction provides a testable innovation linked directly to startup delay. The next phase is to implement the shared simulator and compare policies; the current report establishes the formulation without claiming experimental results.

References and terminology

[1] “CIA–3 Assessment: Reinforcement Learning—Complete Deliverables and Submission Requirements,” course assessment brief, pp. 1–3, 2026. Provided file: RL_CIA_3_Complete_Deliverables.pdf.

[2] Kubernetes Authors, “Horizontal Pod Autoscaling,” Kubernetes Documentation. [Online]. Available: https://kubernetes.io/docs/concepts/workloads/autoscaling/horizontal-pod-autoscale/ . Accessed: Sep. 3, 2026.

[3] C. J. C. H. Watkins and P. Dayan, “Q-learning,” Machine Learning, vol. 8, pp. 279–292, 1992. doi: 10.1007/BF00992698. [Online]. Available: https://www.gatsby.ucl.ac.uk/~dayan/papers/cjch.pdf .

[4] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction, 2nd ed. Cambridge, MA, USA: MIT Press, 2018, chs. 3, 6 and 8. Author resource: https://incompleteideas.net/book/the-book-2nd.html .

Terminology used in the case study

Instance. One identical application-serving unit with fixed compute and memory allocation.

Horizontal scaling. Changing the number of instances that serve the application.

SLA and service target. A service-level agreement specifies commitments. Here, 200 ms is an illustrative performance target, not an external provider guarantee.

95th percentile. A value at or below which 95% of the measured completed-request response times lie.

Policy. A rule that maps an observation to an action; the learned policy chooses among valid actions.

Action mask. An operational filter that excludes actions violating the current capacity or startup rules.

Pending startup. Capacity requested earlier that has not yet been activated under the simulator’s boundary ordering.

Relationship to the presentation

The report retains the presentation’s one-to-five-instance scope, 60-second decisions, five-variable observation, three actions, reward weights, Q-learning selection and predictive extension. It supplies additional simulator detail and measurement conventions needed to implement that proposal consistently. All tables of settings and worked numerical examples are original case-study assumptions rather than empirical findings.